6.2.3 UCS2/UTF-16

16-bit character data. Each 16-bit unit is transmitted with the *b1* "first", up to *b16* "last". (NOTE: at a lower protocol level: in 16-QAM 4 bits are sent in parallel, 64-QAM: 6 bits in parallel, 256-QAM: 8 bits in parallel, 1025-QAM: 10 bits in parallel.) Though nominally big-endian, it is bit-wise little endian (so if each octet is 'reversed' individually (instead of reversing 16-bit units), the encoding is little-endian octet-wise).

- Bits per character: 16 (bits per code unit; a character in planes 1-17 are coded by 2 16-bit code units; the byte order is big-endian if 16-bit units are bit-reversed, little-endian if 8-bit units bit-reversed)
- Pad character: CR (as 0x000D in 16 bits)
- Character table: ISO/IEC 10646 [10]; UTF-16, 1 to 2 hexadecets per character

UTF-16 is an extension of UCS2, plane 0, that allows for representing characters in planes 1-16.

When splitting a message text into submessages (using this encoding), there must be no cut inside of a 16-bit unit (if UCS2 is supported; since the max number of message bytes is even for each submessage both for SMS and CBS, this will not happen), nor between "high surrogate" and a "low surrogate" (the pair encodes for a character in plane 1 to 17; if UTF-16 is supported). (There should be no "surrogate" not in such a pairs; that would be an encoding error.)

Note that for CBS, a 2-letter language tag prefix for UCS2/UTF-16 encoded messages are encoded in the 7-bit 0x00 alphabet in the first two bytes; it should be lowercase, and a three-letter language tag has a special encoding; both should be lower-case. Note that *some* three-letter tags are equivalent to a corresponding two-letter tag.

Furthermore, if any of the 7-bit alphabets 0x10 and onwards can be used for an SMS/CBS message, use of that 7-bit alphabet is preferred, as that can more than half the length (in bytes) of the message, allowing for fewer submessages and allowing a longer (in characters) total message. (Note that alphabets 0x00-0x0F are deprecated for SMS/CBS messages.)

6.2.3.1 UCS to UCS mapping (for both UTF-8 and UCS2/UTF-16)

For messages originally encoded in a UCS encoding, the following mappings should be used, both upon sending (before encoding in a 7-bit encoding, if that is to be used), as well as upon receiving a UCS2 SMS/CBS message:

- Most C0/C1 and NBSP: U+0000-U+0008, U+000E-U+001F, U+007F-U+009A, U+009C-U+00A0: map to U+FFFD (REPLACEMENT CHARACTER, the Unicode version of SUBSTITUTE).
- U+0009 (HT): map to U+0020. NLFs: U+000B, U+2028-U+2029: map to U+000A (LF). U+061C (ALM): map to U+200F (RLM).
- Fixed width spaces: U+2000-U+200B: map to U+FFFD.
- If in the immediate vicinity of an Arabic letter: U+200C (NON-JOINER): map to U+00B7 (use middle dot as non-joiner). U+200D (JOINER): map to U+0640 (use kashida as joiner). Note that U+200D and U+200C have special handling, for or against certain conjuncts or other ligatures, for several Indic scripts, including when converting to 7-bit alphabets (see section A.4).
- Various other characters not suited for SMS/CBS: U+202A-U+202E, U+205F-U+206F, U+D800-U+F8FF, U+FDD0-U+FDEF, U+FEFF, U+FFFC, U+xFFFE-U+xFFFF (where x is in $\{0, \ldots, 10_{16}\}$: map toU+FFFD.
- Map Arabic question mark to reversed question mark… (etc.)

To make texts more readable, we will make some adjustments (corrections) to the Unicode bidi algorithm and to the Unicode line breaking rules below. These adjustments also apply to the 7-bit alphabets (usually after mapping to the UCS upon receiving a message). These adjustments may become redundant, as Unicode evolves.

6.2.3.2 Automatic line breaking improvements

There are a few more things amiss with status quo (2026) for Unicode line breaking (in addition to needing better disambiguation for QU characters).

SP: A non-empty sequence of SP always, with *one* exception, offer a line break opportunity after the sequence. The exception: for convenience, a single SP between two Nd (decimal digits) offers *no* line break opportunity after the space. This exception is due to the commonality of using a space as thousands separator; though PUNCTUATION SPACE (which is not a line-break:SP) would be ideal, but that character is in practice unused. Thus, SP before !?;:» and after « *do* offer a line break opportunity after the SP. There is no special treatment for fr_FR (the only special case locale regarding this) and 'two-part' punctuation, like ?!;:«» (these are the two-part punctuations used in French, the only special cases for the fr_FR locale, note the special space handling is NOT applied to ‹› which are not ‘two-part’ in this sense; but the SMS protocol does not have any language tagging). Use NARROW NO-BREAK SPACE for no-break spaces in this case.

Final punctuation, here ;:,.?! and reverse ,;? and turned ,;, plus CL (mainly }]) and QU resolved to CL): A (commonly mixed) non-empty sequence of these offers a line break opportunity *after* the sequence if there is no LF or eos after it. However, there are two *exceptions*: 1) (numerals with decimal part) a single , or . between two decimal digits (Nd) offers *no* line break opportunity after the comma or full stop, and 2) (some abbreviations) a single letter (L*), followed by a colon or full stop, followed by a single letter, offers no line break opportunity after the colon or full stop. For Brahmic scripts (Indic and more) also ‘danda’ and ‘double danda’ are also final punctuation; but these are rarely used in modern texts.

Solidus (and reverse solidus): A / or \ that is immediately preceded by at least two general category L* characters, immediately succeeded by at least two L* characters, offers a line break opportunity after the / or \. This way there is a line break opportunity when / is used as “or”, as well in (most cases) path names (incl. URLs), which may be quite long without other break opportunities, still without any break in certain abbreviations like n/a, nor in divisions like 16/20.

6.2.3.3 Complex scripts

Brahmic (mostly Indic) scripts (if supported) must be handled specially for rendering; the details of which are beyond the scope of this standard, see the Unicode standard. Further, while most ‘emoji’ characters are isolated small, coloured drawings, many emoji are made from ‘emoji sequences’, multiple emoji characters creating one emoji to display; again, for details, see the Unicode standard.

For the Latin script alphabet (“European”, whether using the alphabet with identifier 0x10 or UCS2/UTF16) most text for “European” languages (and Vietnamese) can be handled be handled by “precomposed” letters in ISO/IEC 10646 (UCS). However, to be able to fit the Latin characters for these languages into one 7-bit alphabet (the 7-bit alphabet with identifier 0x10), it relies on using combining characters, in particular for East European languages and Vietnamese. Similarly for polytonic (traditional, until 1980) Greek; modern (after 1980) Greek orthography is monotonic Greek, and that is covered without the need for combining characters in the 7-bit alphabet for Greek, identifier 0x11. The easiest way to handle that is to upon a UE receiving a message in the Latin (0x10) or Greek (0x11) 7-bit alphabet, is to first convert to ISO/IEC 10646 (equivalent to Unicode in the set of characters encoded) and then apply the Unicode Normal Form C (NFC). When sending from a UE, first apply NFC, then decompose letters that need to be represented in decomposed form in the Latin or Greek 7-bit alphabet. Note that Brahmic (mostly Indic) scripts commonly use combining characters (for all vowel marks, and more), and some base letter plus combining mark have precomposed forms, not all of which are in the 7-bit alphabets.

Also the Hebrew script is counted as a complex script, not only for vowel-point (which are often left out), since it is written and read mostly from right-to-left; mostly since modern numerals are written using digits 0-9, and the numerals are written left-to-right, as is embedded text in an LTR script (like the Latin script). Similarly for the Arabic script (which is used also for some other languages, like Urdu), with the addition that Arabic letters ‘cursively join’ and has some obligatory ligatures.

6.2.3.4 Bidirectional scripts (in particular Hebrew, Arabic)

In case there are any bidi RTL characters in the message, a *readability updated* Unicode bidi algorithm shall be used before display of the message.

Already in the mappings above we noted: U+061C (ALM): map to U+200F (RLM). Further: bidi AL are handled as bidi R, except Arabic punctuation (originally bidi R) which are handled as bidi ON like all other punctuation; all bidi EN and AN are handled as bidi L; all bidi ET, ES, and CS are handled as bidi ON. Without this simplification certain character sequences are reordered in an obfuscatory way, which impedes readability. This simplification is not limited to SMS/CBS; indeed, one should use this simplification generally.

There are further (general) improvements to the bidi algorithm in section 6.2.3.5 and 6.2.3.6 below; also done for readability reasons.

The 'paragraph direction' is LTR for SMS/CBS messages, unless explicitly set to ‘always RTL’ as a user preference setting; no attempt at setting it based on the given text. (Indeed, the bidi controls for that are mapped away for SMS/CBS.) There are no nestable bidi controls in SMS/CBS (or they are not interpreted in the simplified bidi algorithm, in case the mappings above have not been applied); for SMS/CBS bidi controls should be mapped to U+FFFD (see 6.2.3.1). This is a simplification just for SMS/CBS and should not be generally applied.

Furthermore, Arabic is a ‘cursive’ script and Arabic characters thus should be cursively shaped (initial, medial, final, isolated forms), no use of preshaped Arabic letters in SMS/CBS. Both Arabic and Hebrew may have (certain) combining characters applied (like for other scripts as well). Bidi mirroring (when displaying RTL text sections) applies, but (simplified for SMS/CBS) *only* those that can be done via using the data in *BidiMirroring.txt* (Unicode) (note: *not* the comments at the end of that file).

6.2.3.5 Quote mark (line-break:QU) disambiguation for bidi and automatic line break

Quote marks are, unfortunately, handled in a rather strange way in European typographic traditions. Tradition vary depending on region (almost by language, but not quite), and sometimes the same mark is used both for begin quote and end quote, or used in the opposite way compared to how they are used by others. Since we cannot remake European typographic traditions, we will have to do the second-best thing: do an *ad hoc* derivation of whether a quote mark is a start one or an end one. It is based on what is on either side of the ambiguous quote mark: letters, punctuation, space. It will not be perfect, but better than what is currently (2026) done in the Unicode standard for line breaking (which is just plain wrong), and currently complete lack of handling of quote marks for the bidi algorithm.

Modern Khmer, Thai and Lao scripts do not use space between words, but uses European punctuation, including for quote marks. So it will be common that rules 1 and 2 below will not be able to disambiguate. However, for these scripts the use of quote marks seem to follow their use in English or French. I.e. one can use the Pi and Pf general category to see if they are begin or end (which one *cannot* do for European languages, except English and French…; but we do not use that for English nor French, especially since SMS do not have any means of language tagging). That is handled by rule 4 below, which is similar to the current (2026) handling in the Unicode line-breaking algorithm.

Note that Chinese and Japanese (and Korean?) conventionally use other quote marks that are not ambiguous.

6.2.3.5.1 QU ad hoc disambiguation (for line-break and bidi)

In the Unicode line breaking standard there is the line break property QU for marking ambiguous quote mark symbols. These should be resolved (ad hoc) to either line-break:OP (opening; and bidi-bracket:o, opening) or to line-break:CL (closing; and bidi_bracket:c, closing). If the ambiguous quote mark cannot be resolved (ad hoc) as opening or closing are to be regarded as line-breaking:SY (symbol; and bidi-bracket:n, neither). {QU} refers to the character that has the (line-break) property QU (etc.), bos is beginning of string, eos is end of string, dash

refers to any of the Unicode dashes. There is no HT (nor VT) in the 3GPP alphabets, even for UCS2/UTF-16 they are mapped away. If a line of text (before automatic line breaking) has one or more QU characters:

1. ({OP}|bos|SP|LF|FF|dash){QU}(NNBSP|{OP}|L*|N*|{QU}|INV.Q.M.|INV.E.M.) → regard the central {QU} as (line-break) {OP} and (bidi-bracket) {o}.
2. (CL|L*|N*|{QU}|[,;.:?!]|NNBSP){QU}({CL}|eos|SP|LF|FF|[,;.:?!]|dash) → regard the central {QU} as (line-break) {CL} and (bidi-bracket) {c}. ELLIPSES are regarded as FULL STOP here. REVERSE question mark, reversed or turned comma, reversed or turned semicolon are regarded as ?,; here.
3. Repeat the above two on the string until no match.
4. Remaining QU (not resolved by rules 1 or 2):
   (a) A QU that is Pi, regard it as line-break OP as well as bidi-bracket:o, open.
   (b) A QU that is Pf, *except* ’ between Latin script letters, regard it as line-break CL as well as bidi-bracket:c, close.
   (c) ASCII quote marks " ', and ’ between Latin script letters: regard as line-break SY and bidi-bracket n.

No line-break:QU remains to disturb the finding of line break opportunities. And a begin quote mark should rarely be mistaken for an end quote mark (and vice versa), given the author follows punctuation and space conventions. Likewise for bidi, where ambiguous quote marks are in the bidi algorithm just seen as symbols, not marking begin or end of a quote. The ad hoc derivation does no semantic analysis of the text, and people do not always follow writing conventions. The best we can do in an ad hoc analysis is assuming that writing conventions are followed sufficiently well.

6.2.3.6 Clarifying, and extending to QU, bracket matching for bidi processing

Before bidi processing, map all U+0086, U+0087, U+0096, U+0097 to SUBSTITUTE (or REPLACEMENT CHARACTER, technically equivalent (though not formally so in Unicode), but REPLACEMENT CHARACTER is better supported), as specified in section 6.2.3.1. Normally, there should not be any of those characters anyway.

To get bracket matching to work well, we will use four format control (Cf) characters, U+0086, U+0087, U+0096, U+0097. We let them all have bidi-mirroring:Y and the first two are each other’s mirror, likewise the last two are each other’s mirror. The first two are bidi-bracket:o and c respectively, and the last two are bidi-bracket:wo and wc respectively; wo and wc are new bidi-bracket values that we will use below.

All bracketing analysis for bidi requires full proper nesting of the bracketing. That includes also nestable bidi controls (like LRI---PDI, and several others). They must nest in “one pass”; not just in separate “passes”. We will insert U+0086, U+0087, U+0096, U+0097 (and nestable bidi control characters) as appropriate to achieve this full proper nesting; U+0086, U+0087, U+0096, U+0097 act as ‘stand in’ brackets for begin-end bracket matching, but are nominally invisible format control characters. We also need to regard all begin-bidi-controls as bidi-bracket:c, and all end-bidi-controls as bidi-bracket:c. (There also needs to be an ad hoc guess as to what a missing begin-bidi-control should be; but as mentioned, we will gloss over the nestable bidi controls here, since they are not allowed in 3GPP messages.)

(We will here skim over how to handle the nestable bidi controls, since these are not allowed in 3GPP messages (mapped to REPLACEMENT CHARACTER). That is because these controls can make a message read completely differently that what it actually says; in grave cases it can appear benign but actually say something hostile. That is something we do not wish to encourage, even though we cannot make it completely ‘fool proof’ (but we do not allow any erasing backspace characters in messages either, another thing that would be problematic; similarly, CR is mapped to LF when CR is not a filler).)

The Unicode bidi algorithm (Unicode technical standard 9, section 3.1.3 Paired Brackets) loosely assumes that there is bracket ‘pairing’, presumably (as of 2026, not clearly stated in the bidi algorithm) trying to match with a ‘mirror’ bracket character (as in *bidi-mirroring.txt* for Ps and Pe characters); presumably (again) including nest counting ignoring all *other* Ps/Pe when looking for a matching ending for a given Ps, as well as ignoring all QU characters (corrected in next paragraph here), going from beginning of string, outer to inner.

Now that the QU (ambiguous quote marks) have an (ad hoc) disambiguation, we can do a similar begin/end matching for them as for the paired brackets. Note that strict nesting applies, just like for the Ps/Pe brackets for this matching, and done in the same “pass” as the Ps/Pe bracket matching.

- Begin: ('|‘|’|,|‘) that is disambiguated as OP/o, matching end: ('|‘|’|,|‘) that is disambiguated as CL/c. Note that , is *not* a COMMA, ,, even though it looks exactly like one (and in origin were the same); it is a SINGLE LOW 9 QUOTATION MARK. For (bos|LF|SP)COMMA(L*|N*): the COMMA may be mapped to SINGLE LOW 9 QUOTATION MARK during input. The ‹‘› is never preferred.
- Begin: ("|“|”|„|‟) that is disambiguated as OP/o, matching end: ("|“|”|„|‟) that is disambiguated as CL/c. Note that „ is *not* a double COMMA, ,,, even though those look similar, just a small spacing difference; it is a DOUBLE LOW 9 QUOTATION MARK. For (bos|LF|SP)COMMA-COMMA(L*|N*): the COMMA-COMMA may be mapped to DOUBLE LOW 9 QUOTATION MARK during input. The ‹‟› is never preferred.
- Begin: (‹|›) that is disambiguated as OP/o, matching end: (‹|›) that is disambiguated as CL/c.
- Begin: («|») that is disambiguated as OP/o, matching end: («|») that is disambiguated as CL/c.

Note that the (REVERSED/LOW) DOUBLE PRIME QUOTATION MARKs as well as the CJK quotation marks are not QU.

Note that in an actual text, brackets might not always have open-close matching. The brackets that did not get a match: a) if it is a bidi-bracket:o, after a following sequence of (L*|N*)* substring insert a (matching!) U+0087, b) if it is a bidi-bracket:c, before a preceding (L*|N*)* substring, insert a (matching!) U+0086.

Since bidi is processed per paragraph, bracketing that spans multiple paragraphs will not be correctly handled, neither in the original Unicode bidi algorithm, nor in the modification specified here.

6.2.3.7 Improved sentence structure conservation for the bidi algorithm

There is more bracketing structure in texts that are not given by explicit begin/end ‘brackets’ (of various kinds). And that is phrase (or sentence) structure. The phrases do not have a begin punctuation, they only have end punctuation: ,;.:!?. If we do not deal with the phrases as bracketed, we will have word ordering issues if a mix of LTR and RTL characters occur near one of these phrase ending punctuation, ordering issues that are basically the same as when not dealing with begin/end bracketing, as was the case with the bidi algorithm initially.

So we have to mark begin/end of phrases as well, and deal with them in the same way as explicitly begin/end bracketed parts of the text. To that end we will, when needed, insert a special brackets for phrases; we will use U+0096 (but often SPACE that is after a phrase ending punctuation is used) for begin phrase bracket, and U+0097 when there is no explicit phrase ending (i.e. no ,;.:?!, note that not all instances of these are wc).

Introduce two new bidi-bracket categories: weak-open (wo) [for U+0096 and some SPACE] and weak-close (wc) [for U+0097 and ,;.:?!), dynamically introduced (i.e. *not* in character property file except for U+0096 and U+0097). Introduce them as follows (note that bidi is processed per ‘paragraph’, and here we use LF as paragraph separator):

1. For bos that is not just before eos (of paragraph), or between a bidi-bracket:o char and a bidi-bracket:n char: insert a U+0096 (wo) (if there is an LF there, the LF can get the bidi-bracket:wo, avoiding the insertion).
2. For ,;.:!? (and reverse ,;? and ellipses …) *with* a SPACE just after it: set the punctuation itself to be bidi-bracket:wc (weak close) and the space just after it is as bidi-bracket:wo (weak open); avoiding inserting <U+0097, U+0096>.
3. For ,;.:!? (and reverse ,;? and ellipses …) *with* a bidi-bracket:c or eos (of paragraph) just after, set the punctuation itself to be bidi-bracket:wc (weak close).
4. Between a bidi-bracket:n char and a bidi-bracket:c char: insert a U+0097 (bidi-bracket:wc).
5. For eos that follows a bidi-bracket:n: insert a U+0097 (bidi-bracket:wc).

Note that *other* ,.:;?! are treated just as any other symbol (bidi-bracket:n), and so is inverted ?!. Thus, inside a number, inside an abbreviation or inside a filename or URL, there is no wc/wo bracketing.

Matching wo-wc are matching pairs (they do not nest within an o-c while skipping inner o-c) *within* each matching o-c pair (strong bidi-brackets), *skipping* 'inner' o-c parts for each o-c pair part. ((There should never be any unmatched wo or wc character when inserted according to the rules… CHECK THE RULES AGAIN!))

Examples; the string starts with "It", { is wo, } is wc, [is o,] is c, the first { (wo) is through an inserted U+0096:

```
 It was August 3, 500 birds in the sky.
{nnnnnnnnnnnnnnn}{nnnnnnnnnnnnnnnnnnnn}
```

Here the first and second { are via inserted U+0096, and the last two } are through inserted U+0097.

```
 It said "number 3, 500£" as if it was an auction
{nnnnnnnn[{nnnnnnnn}{nnnn}]nnnnnnnnnnnnnnnnnnnnnnn}
```

(Recall that we did an ad hoc determination if quote marks were opening or closing brackets.) Now assume that the English text, except the word 'number', instead was in a bidi script.

Without the wo-wc bracketing the first example would be displayed as:

```
.yks eht ni sdridb 3, 500 tsuguA saw tI
```

I.e. August 500, and just 3 birds… *With* the wo-wc bracketing, we get the correct reading:

```
.yks eht ni sdridb 500, 3 tsuguA saw tI
```

The second example illustrates how strong and weak bracketing properly nests.

6.2.3.8 Bidi ON → bidi L or bidi R for bidi-bracket:c and bidi-bracket:wc and for word adjacent symbols

During the initial phases of the bidi algorithm (but after bidi-bracketing analysis), characters that are ON (other neutral) are assigned a direction (L or R depending on the surrounding characters). This step is augmented with the following rules for bidi-brackt:c (close) and bidi-bracket:wc (weak close).

1. Word adjacent ON (symbols including punctuation): An ON char, that is not a space, nor a bracketing punctuation, nor a weak bracketing punctuation (,;.:!?), but has a bidi L or bidi R character (typically a letter or a digit, but could be a symbol given a strong direction by this rule) on either side, but an ON or bos or eos on the other side, is assigned the strong direction it has on one side. Repeat until no change.
2. Bidi-bracket:c character: *When* a bidi-bracket:o character is given a strong direction (by other rules), the *matching* bidi-bracket:wc character gets the same direction directly (no other rules)
3. Bidi-bracket:wc character: *When* a bidi-bracket:wo character is given a strong direction (by other rules; the U+0096 and U+0097 are bidi:ON initially) the *matching* bidi-bracket:wc character gets the same direction directly (no other rules). For any sequence of ,.;:!? (may be mixed; includes reverse ,;?) immediately before it, assign those ,.;:!? to the same strong direction as the ,.;:!? that is bidi-bracket:wc.

Note that point 1 is to be done prior to other ON resolution rules, while points 2 and 3 are to be done during other ON resolution rules.

6.2.3.9 Simple tables via 3GPP-specific use of IS1..IS4

The "INFORMATION SEPARATORS" (IS1..IS4) are private use control codes for, as the name says, some kind of "information (data) syntax punctuation/'separators'". The syntax for their use is not given by ISO/IEC 6429, but they derive from the instance in ASCII and EBCDIC of "UNIT SEPARATOR" etc. (the original, and better, CSV format). However, that is just the ASCII 'instantiation' of the (later defined) IS1..IS4. Other data protocols may define a different syntax using these characters as data punctuation, which we will do here. The original CSV format (as well as the currently popular one), do give a tabular data syntax, but it is assumed that

the entirety of the given data is/are the table(s). There is no way of embedding the table in other text, that would need some kind of extension or modification.

For short messages it may be useful to embed small, and very simple, tables into a message. We cannot use the current CSV format (extended), since that uses characters that are used also for the data proper. But we can use a modified version of the original CSV format, using IS1…IS4, since they are not used for running text, but a variation of the syntax so that the tables can be embedded in the text.

For 3GPP 23.038 use, we define (non-exclusively), in the SS3 table for the 7-bit alphabets, this mapping to UCS:

0x1B1B43	U+001F	INFORMATION SEPARATOR ONE (use as BEGIN TABLE)
0x1B1B44	U+001E	INFORMATION SEPARATOR TWO (use as CELL SEPARATOR)
0x1B1B45	U+001D	INFORMATION SEPARATOR THREE (use as ROW SEPARATOR)
0x1B1B46	U+001C	INFORMATION SEPARATOR FOUR (use as END TABLE)

So now we have both for the 7-bit alphabets and for UCS (UTF-16 and UTF-8), a way of specifying simple embedded tables. The tables are block formatted (not inline), with single line border for all cells, horizontally centered. The cells are laid out left to right if the paragraph direction is L (default), but right to left if the paragraph direction if R (manually set by user).

For simplicity:

- No tables in tables.
- Single nominal width border line drawn for all cells.
- The table is horizontally centred in the available space (the table should be ‘draggable’, in some way, horizontally if the table is wider than the available space).
- No LF in tables, nor any automatic line break in table cells.
- Each table has a block layout, i.e. BEGIN/END TABLE imply line break.
- There must be the same number of cells in all the rows in one table.
- No cell spanning over rows nor columns.
- Formally, no header row, nor header column; though formatting, like bold can be used; any formatting reverts to default at cell end.
- There should be at most 5 columns.
- There should be at most 10 rows.
- The rows should have a max width of 50 em (including cell borders).
- Cell width is the max actual width of the texts in the column (including cell border).
- The cell content is left adjusted if the paragraph direction is L, it is right adjuste if the paragraph direction is R. Bidi processing (if applied) is applied to each cell individually.

6.2.3.10 CSI control sequence compression

See section D.3.3.